5-5 · 파티셔닝과 셔플링 최적화 · 실습 1.5H

repartition/coalesce와 셔플 관찰

파티션 수를 바꿔 가며 시간과 셔플을 잰다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① getNumPartitions() 로 4 · 8 · 2 · 2 를 확인한다

② 4040 Jobs 탭에서 셔플 있는 잡과 없는 잡을 가른다

③ AQE 를 켜고·끄고 shuffle.partitions 200 · 8 을 잰다

④ 스큐를 만들어 Task 시간 Max 와 Median 을 비교한다

잰 값(파티션 수·시간·파일 수)은 다음 1H 실측표에 그대로 적습니다

실습 흐름

여섯 단계 — PySpark 셸 하나에서 끝까지

bash lake-pyspark.sh 로 셸을 띄우고 localhost:4040 Jobs 탭을 열어 둡니다

1·2. 파티션	3. 셔플 관찰	4. 셔플 파티션	5. 스큐	6. 파일 수
수 4·8·2·2 조각별 행 수	Jobs 탭 explain()	AQE 켜고·끄고 200 vs 8	키 0 에 90% Max / Median	test-narrow 4 test-one 1

1·2단계

파티션 수와 조각별 행 수 — 실측

명령	실측
spark.range(1, 10_000_001)	기본 4 (워커 둘 × 코어 2)
repartition(8)	8
coalesce(2)	2
coalesce(16)	여전히 2 — 늘릴 수 없다
repartition(4) 분포	2500000 × 4 — 고르게
coalesce(2) 분포	[5000000, 5000000]
3조각 → coalesce(2)	[6666667, 3333333]

coalesce 는 셔플 없이 옆 조각을 붙이기만 — 고른 4조각은 둘씩 고르게, 3조각은 한쪽이 두 배

3단계

셔플을 Jobs 탭과 explain() 에서 관찰

1. **(A) filter 후 test-narrow 로 저장**
   잡 하나 · Stage 1/1 · Task 4/4 — 셔플 없음

2. **(B) groupBy(bucket).count().collect()**
   AQE 가 잡 둘로 — 4/4 Shuffle Write 2.4 KiB → 1/1 Read 2.4 KiB

3. **explain() 으로 같은 판정**
   둘째에만 Exchange hashpartitioning(b#59L, 200)

애플리케이션 UI

localhost:4040

Jobs 탭

> **(A) 를 count() 로 끝내면 Stage 가 둘 — count 도 마지막에 조각별 개수를 모은다. 그래서 저장으로 끝낸다**

4단계

bench(n) — 셔플 파티션 수를 바꿔 시간 재기

그룹은 10개뿐 — 기본값 200 이 이 데이터에 맞는지 봅니다

AQE 확인	켠 채로	AQE 끄기	끈 채로	원복
adaptive.enabled true	200 · 8 각각 시간	enabled false	200 · 8 다시 시간	AQE true partitions 200

4단계 — 일부러 헛짚기

AQE 를 켠 채로는 200 vs 8 차이가 작다

AQE 켜짐 (기본)

실행 중에 빈 조각을 합쳐 준다 — 차이가 작다

- 200 → 1.34초
- 8 → 0.96초
- 「줄이면 빨라진다」가 안 보인다

VS

AQE 꺼짐

Task 200개 중 190개가 빈 태스크

- 200 → 4.59초
- 8 → 0.90초
- Stages 탭에서 Task 200개 확인

5단계 — 일부러 느리게 만들기

스큐 — 90% 가 키 0 에 몰리면

1. **repartition(4, "k") 분포**
   스큐 [10000000] · 고른 분포 [2800000, 2400000 × 3]

2. **sha2 를 시킨 뒤 count() 시간**
   2.20초 · 3.18초 — 실행마다 들쭉날쭉, 그래서 UI 를 본다

3. **Stages → Summary Metrics Duration**
   스큐 Max 1.1~1.8초 · Median 0.01~0.06초, 고른 쪽 0.58 · 0.55초

Max 가 Median 의

몇 배 넘으면

스큐 의심

> **groupBy().count() 는 조각마다 미리 세어 보내 스큐가 잘 안 드러난다 · 해결책 솔팅은 개념만**

6단계

저장 직전 파티션 수 = 파일 수

1. **coalesce(1) 로 test-one 저장**
   3단계 (A) 의 test-narrow 와 같은 데이터

2. **새 터미널에서 mc find ... | wc -l**
   test-narrow: 4 · test-one: 1

3. **grep 으로 세면 command not found**
   mc 이미지 안에는 grep 이 없다 — mc find 와 wc -l 로 센다

작은 파일 문제

4주차

rewrite_data_files

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. 4 · 8 · 2 · 2, 3조각 → coalesce(2) 는 [6666667, 3333333]
2. (A) Stage 1/1 · (B) Shuffle Write/Read 2.4 KiB
3. explain() 둘째에만 Exchange hashpartitioning
4. AQE 꺼짐에서만 200 이 8 보다 눈에 띄게 느리다
5. 스큐는 Task Max 가 Median 의 수십 배
6. test-narrow 4 · test-one 1

5-5 · 실습 1.5H 마무리

다음은 배치 스크립트로 데이터 마트 (1H)

셸을 exit() 로 나온 뒤, 지금까지의 가공을 mart.py 한 파일로 묶어 spark-submit 으로 돌립니다 — 5주차 산출물입니다

- ✓ **exit() 로 셸의 코어 4개 놓아주기**
- ✓ **히어독으로 mart.py 작성 · spark-submit**
- ✓ **두 번 돌려 멱등 확인**
- ✓ **날짜 인자 · mart.log · 종료 코드**

가져갈 것

4·8·2·2 · 200 vs 8 시간 · 파일 4·1